Phase 7 — Floor-Supervisor View

LineTwin · Accenture Innovation Challenge 2026 · Round 2 · Problem Track 4 "DigitalTwin.ai"

Purpose

Build the real-time dashboard a floor supervisor actually watches: a zone-grouped 30-station grid, the live bottleneck callout, trend charts, and a perturbation control — on the restored Accenture palette, verified live in a browser rather than assumed correct from the code.

This phase's own standing rule — nothing counts as done until it has been watched happening in a browser — caught **two real bugs** a code read alone would have missed, one of them a second, independent way to deadlock the entire server that the Phase 6 fix did not cover. Both are documented in full below, including a wrong turn along the way, because the wrong turn is as instructive as the fix.

What was built

Artefact	What it does
`web/index.html`, `web/styles.css`, `web/app.js`	Vanilla HTML/CSS/JS, no framework, no build step. Accenture brand tokens restored (`#A100FF` / `#6A00B8` / `#F2F2F2` / `#E8590C` + `#FBD7BF` / `#C81E3A` + `#F6C6CE` / `#1E8E3E` / `#1A1A1A` / `#595959`)
`web/vendor/uplot/`	uPlot 1.6.32 vendored from its actual npm release (`uPlot.iife.min.js`, `uPlot.min.css`) with its MIT `LICENSE` file, not CDN-loaded
`src/twin/api/routes.py` (extended)	Static files mounted **last**, after every API route — proven, not just asserted, by a test that hits both `/healthz` and `/app.js` against the same app
`tests/test_api.py` (+1 test)	Static-file serving + API-priority-over-mount check
`tests/test_engine.py` (+1 test)	Regression test for the second deadlock found this phase (below)

The dashboard shows: a zone-grouped station grid (Body/Paint/Final) with per-station state, queue depth, cycle time and its provenance tag; a bottleneck callout with confidence, runner-up, mode decomposition, and Bottleneck Walk phrasing; two uPlot trend charts (throughput, WIP); a perturbation control (station + multiplier + Apply) wired to `POST /api/twin/control`; and a vitals bar with a real `EventSource.readyState`-driven lamp, Kill/Resume, and Restart.

Bug 1: the dark-station provenance tag lied

Live in the browser, **S07** (a genuinely uninstrumented station) rendered as `cycle –` with a **green "observed" pill** — exactly the failure mode `docs/CITATIONS.md` and `contracts.py`'s own design rationale exist to prevent: presenting an absence as a measurement. The bug was in `app.js`, not the data: `TaggedValue.missingness` is supposed to be authoritative when a value is absent, but the frontend rendered `ct.source` unconditionally regardless of `missingness`. `engine.py` correctly set `source=OBSERVED` as a moot default alongside `missingness=MISSING` (there is no "n/a" `ValueSource` value in the frozen contract), and the UI never checked which field to trust.

Fixed: the tag now reads `missingness` first, only falling back to `source` once `missingness === "present"`. Re-verified live: S07 now shows a grey, italicized **"missing"** pill.

Bug 2: a second, independent way to deadlock the entire server

Manual browser testing found the server pinned at 100% CPU again, every request — including `/healthz` — hanging indefinitely, after: apply a large perturbation (9× on one station, producing a big cascade of BLOCKED stations), let it run a couple of seconds, then close and reopen the SSE stream (the dashboard's own Kill/Resume buttons). This is a **different** bug from Phase 6's — that one is already fixed and its regression test still passes.

The wrong turn, kept in the record rather than erased. Chrome's network panel showed `net::ERR_INCOMPLETE_CHUNKED_ENCODING` on the torn-down connection, and `curl -N` never reproduced the deadlock even after 15 rapid connect/disconnect cycles against an identically-perturbed server — only a real browser `EventSource` triggered it. That evidence pointed at `sse.py`'s `await request. is_disconnected()` poll (called up to 8×/second per connection; Starlette's implementation reads a message off the request's ASGI receive channel inside an already-cancelled `CancelScope`, a more invasive operation than it looks). The poll was removed, relying instead on FastAPI/Starlette's documented behavior of cancelling a route's generator on disconnect.

That fix did not work. The identical deadlock reproduced immediately on the next attempt.

Root-caused for real with a stack trace, not another guess. `py-spy` needs root on macOS, which was not used without asking; `faulthandler.register(signal.SIGUSR1, all_threads=True)` needs no elevated privileges and was added to an instrumented copy of the server script instead. Reproducing the hang and sending `SIGUSR1` dumped the exact stack:

```
scipy/integrate/_quadpack_py.py:628 in _quad
scipy/stats/_continuous_distns.py:12461 in _single_cdf
statsmodels/sandbox/stats/multicomp.py:184 in get_tukey_pvalue
statsmodels/sandbox/stats/multicomp.py:1306 in tukeyhsd
statsmodels/stats/multicomp.py:43 in pairwise_tukeyhsd
diagnostic/bottleneck.py:169 in significance_annotation
sim/engine.py:250 in _publish_snapshot   <-- called synchronously
```

An extreme perturbation makes the active-period distributions for different stations extremely separated. `statsmodels`' Tukey-Kramer p-value goes through `scipy`'s studentized-range survival function, computed by **adaptive numerical integration** — which becomes pathologically slow for extreme, widely-separated inputs. That is synchronous, CPU-bound Python code with no `await` points, called directly from the tick loop; it blocked the **entire single-threaded event loop**, including every HTTP handler, for as long as the integration ran.

Fixed: `bottleneck = await asyncio.to_thread(diagnose, views)` in `engine.py`. This is the correct tool specifically *because* the computation's duration is unbounded and can genuinely reach multiple seconds — the opposite case from Phase 8's live risk-scoring inference (a ~microsecond XGBoost predict, where `to_thread`'s own dispatch overhead would dominate and offloading would be the wrong call). Both are documented together in `engine.py` so the distinction isn't lost.

Re-verified against the exact failing sequence, not a simplified stand-in: the identical 9× perturbation, wait, kill, wait, resume — `/healthz` responded in 10 ms *during* the cascade and in 74 ms immediately after resume, versus complete, permanent unresponsiveness before the fix. Repeated with zero delay between kill and resume (the harder case) and confirmed clean.

Regression test, proven to catch the regression, not merely written: `tests/test_engine.py::test_a_slow_diagnose_does_not_block_the_event_loop` runs a monkeypatched `diagnose` that sleeps synchronously for 0.4s, alongside a "canary" coroutine ticking every 10ms, and asserts the canary accumulates enough ticks to prove the event loop stayed responsive. Verified both ways: passes in 8.56s with the fix in place; reverting the fix (`bottleneck = diagnose(views)` without `to_thread`) made the same test **not complete within 120 seconds** — direct proof the test catches exactly this regression, not a coincidence of timing.

Live verification performed

- **Golden path:** applied a 9× perturbation to S05 via the real control panel — the cascade (S04, S07, S08 turning BLOCKED, S05's own cycle time jumping from 38.5s to 363.5s) was visible within approximately 0.75 seconds, comfortably inside the 2-second requirement.
- **Kill:** lamp reddens, every vital freezes (`tick` / `seq` identical across a 2-second wait), station cards dim, Kill disabled/Resume enabled.
- **Resume:** lamp greens, ticks resume advancing, cards return to full opacity.
- **Bottleneck correctness under load:** with S05 at 9×, the dashboard correctly named S05 as the bottleneck (not a neighbor), with the outline highlight landing on the right card and the station grid's BLOCKED/STARVED coloring matching the true upstream/downstream cascade pattern exactly.
- Confirmed the frontend's provenance-tag fix and the deadlock fix together under the same live session, not in isolation from each other.

Exit criteria

Criterion	Status
Zone-grouped 30-station grid on the Accenture palette	Met
Bottleneck callout with mode decomposition + confidence annotation	Met
uPlot vendored with its MIT licence; throughput + queue/WIP charts live	Met
Vitals bar off real `EventSource.readyState`; Kill/Resume	Met
Perturbation visible within 2 seconds, verified live	Met — ~0.75s observed
Latency asserted in a test, not eyeballed	Met — `test_control_then_state_reflect_the_change_live` (Phase 6) plus this phase's live re-verification
Static files served without shadowing any API route	Met — proven by `test_frontend_static_files_are_served_without_shadowing_the_api`
No P0 defect survives to the next phase	Met — both bugs found this phase were root-caused (one with a real stack trace, not a guess) and regression-tested

98/98 tests passing (96 from Phases 1–6 + 2 new this phase).

Next

Phase 8 — Predictive Layer. The offline benchmark model (Model A, real public data, a deliberate SMOTE failure case) and the live risk scorer (Model B, monotone XGBoost, isotonic calibration, a mandatory single-feature baseline comparison) — plus rolling-horizon bottleneck prediction. Given this phase's findings, any new per-tick computation added here gets checked for the same class of problem before it ships: does its worst-case cost matter, and if so, does it run off the event loop.